

Neural Recruiter

Intelligent Candidate Discovery & Ranking

Track 01 | Data & AI Challenge 2026

Beyond Keywords. Beyond Filters. Pure Relevance.

The Problem

Recruiters lose hidden talent in the noise.

- Keyword matching misses contextual fit
- Behavioral signals are invisible to traditional filters
- Manual screening of 100K+ profiles is impossible
- Career trajectory and intent remain hidden

We built an AI brain that sees what humans see — but at scale.

Our Solution: 4-Stage Pipeline

Stage 1: Honeypot Detection — Filter fake profiles & timeline paradoxes

Stage 2: Tier Gating — Suppress juniors if role demands mid/senior

Stage 3: Weighted Scoring — Apply 8 signals; compute 0-1 scores

Stage 4: Penalties & Multipliers — Boost strong matches, penalize weak fit

Result: Deterministic, explainable top 100 shortlist in 60-90 seconds on CPU

8 Signals Driving Relevance

- **Career Progression (20%)** — Role growth, seniority climb, lateral moves
- **Skills Match (22%)** — Semantic + exact skill alignment to JD
- **JD Fit (15%)** — Deep semantic understanding of job requirements
- **Title Tier (15%)** — Seniority alignment & tier gating
- **Experience (10%)** — Years in role, depth, breadth
- **Behavioral (10%)** — Response rates, activity patterns
- **Location (4%)** — Geography relevance, mobility
- **GitHub Signal (4%)** — Developer activity, contribution frequency

Total: 1.0 (100% weighted sum)

Pipeline: Stage by Stage

Stage 1 | Honeypot & Timeline Detection

Filter out fake profiles & resume paradoxes (graduation dates, conflicting timelines)

Stage 2 | Tier Gating

Suppress Tier C (junior) candidates if role demands Tier A/B (mid/senior)

Stage 3 | Weighted Scoring

Apply 8 signals with configurable weights; compute 0-1 normalized scores

Stage 4 | Multipliers & Penalties

Boost strong matches, penalize weak fit; finalize top 100 shortlist

Results & Performance

100,000 candidates processed in 60-90 seconds

Zero external dependencies (pure Python 3.10+)

Fully deterministic & reproducible

Top 5 Candidates (Sample Output)

- **#1:** Senior ML Engineer (7.2y) | Score: 0.9857
- **#2:** Search Engineer (7.6y) | Score: 0.9681
- **#3:** Senior NLP Engineer (7.8y) | Score: 0.9666

Output Format: CSV with candidate_id, rank, score, reasoning

Technical Stack

Backend Ranking Engine

Pure Python 3.10+ | ~500 LOC | Zero external dependencies | Multi-threaded processing

Frontend Sandbox

React 19 + TypeScript | Vite 6 | TailwindCSS 4 | Live weight calibration & export

Reproducibility

`python rank.py --candidates <dataset> --out submission.csv`

Runs on CPU. No GPUs, no cloud APIs, no rate limits. Fully offline.

Why This Approach Wins

- **Semantic Understanding:** Beyond keywords to true relevance
- **Explainable Ranking:** Every candidate has clear reasoning
- **Configurable Weights:** Adjust 8 signals for different roles
- **Fast & Deterministic:** Runs in under 2 minutes on 100K profiles
- **Production-Ready:** No ML framework bloat, pure algorithmic power
- **Scalable:** Proven on 100K+ candidates; runs locally on CPU

The result: Recruiters get a curated, ranked shortlist they can trust.